



Temporal Blood Tracking - 3 Cholec80 Videos

Complete Pipeline: Detection → Tracking → Peak Analysis

This notebook:

1.  Loads trained UNet++ model from Video01 training
2.  Applies to first 3 Cholec80 videos (raw surgical videos)
3.  Generates blood masks for all frames
4.  Visualizes BEFORE/AFTER for sample frames (all 3 videos)
5.  Calculates blood area per frame (industry-grade)
6.  Plots blood area vs time (all 3 videos compared)
7.  Finds peaks (maximum bleeding events)
8.  Shows peak frames with masks
9.  Generates comprehensive statistics

Model: UNet++ trained on Video01 (from previous notebook)

Input: Cholec80 video01.mp4, video02.mp4, video03.mp4

Output: Temporal blood curves, bleeding events, peak visualizations

```
In [1]: # Cell 1: Install Additional Dependencies
!pip install -q opencv-python-headless
!pip install -q scipy # For peak detection
!pip install -q scikit-image # For morphological operations

print( Additional dependencies installed!)
```

 Additional dependencies installed!

```
In [2]: # Cell 2: Imports
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from tqdm import tqdm
import json
import pandas as pd
from collections import defaultdict

# PyTorch
```

```

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# Segmentation Models
import segmentation_models_pytorch as smp

# Albumentations
import albumentations as A
from albumentations.pytorch import ToTensorV2

# Signal processing
from scipy.signal import find_peaks, savgol_filter
from scipy.ndimage import gaussian_filter1d

# Image processing
from skimage import measure, morphology

import warnings
warnings.filterwarnings('ignore')

# Device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"🖥️ Using device: {device}")
if torch.cuda.is_available():
    print(f"    GPU: {torch.cuda.get_device_name(0)}")

print("✅ Imports complete!")

```

```

🖥️ Using device: cuda
GPU: NVIDIA H100 80GB HBM3
✅ Imports complete!

```

```

In [4]: # Cell 3: Setup Paths and Configuration (CORRECTED)
from pathlib import Path

# CORRECTED PATHS based on your directory structure
CHOLEC80_DIR = Path("cholec80/videos") # Cholec80 video files
CHOLECSEG80_DIR = Path("cholecseg8k_raw") # CholecSeg8k segmentation dataset
MODEL_PATH = Path("video01_blood_segmentation/models/best_model.pth") # From
OUTPUT_DIR = Path("temporal_blood_tracking")

# Create output directories
OUTPUT_DIR.mkdir(exist_ok=True)
(OUTPUT_DIR / "masks").mkdir(exist_ok=True)
(OUTPUT_DIR / "visualizations").mkdir(exist_ok=True)
(OUTPUT_DIR / "results").mkdir(exist_ok=True)

# Configuration
CONFIG = {
    'video_ids': [1, 2, 3], # First 3 videos
    'fps': 1, # Extract 1 frame per second
    'image_size': 512,
    'batch_size': 8,

```

```

    'threshold': 0.5, # Blood detection threshold
    'min_blood_area': 100, # Minimum pixels to count as blood
    'peak_prominence': 0.2, # Peak detection sensitivity (0-1)
    'peak_distance': 10, # Minimum frames between peaks
    'smooth_window': 11 # Smoothing window size (odd number)
}

print(f"📁 Cholec80 directory: {CHOLEC80_DIR}")
print(f"📁 CholecSeg directory: {CHOLECSEG_DIR}")
print(f"📁 Model path: {MODEL_PATH}")
print(f"📁 Output directory: {OUTPUT_DIR}")

print(f"\n⚙️ Configuration:")
for key, value in CONFIG.items():
    print(f"    {key}: {value}")

# Verify paths
print("\n" + "="*60)
print("PATH VERIFICATION")
print("="*60)

if not MODEL_PATH.exists():
    print(f"\n❌ ERROR: Model not found: {MODEL_PATH}")
    print(f"    Please train the model first using the Video01 notebook!")
else:
    print(f"\n✅ Model found: {MODEL_PATH}")

if not CHOLEC80_DIR.exists():
    print(f"\n❌ ERROR: Cholec80 directory not found: {CHOLEC80_DIR}")
    print(f"    Searching for videos...")

# Try to find videos
possible_paths = [
    Path("cholec80/videos"),
    Path("data/cholec80/videos"),
    Path("videos"),
    Path("cholec80"),
    Path("data/videos")
]

for path in possible_paths:
    if path.exists():
        videos = list(path.glob("*.mp4"))
        if len(videos) > 0:
            print(f"    ✓ Found {len(videos)} videos in: {path}")
            CHOLEC80_DIR = path
            break
else:
    # Check how many videos we have
    videos = list(CHOLEC80_DIR.glob("*.mp4"))
    print(f"\n✅ Cholec80 directory found: {CHOLEC80_DIR}")
    print(f"    Total videos: {len(videos)}")

```

```

# Show available videos
if len(videos) > 0:
    print(f"\n📺 Available videos:")
    for vid in sorted(videos)[:10]:
        size_gb = vid.stat().st_size / (1024**3)
        print(f"    - {vid.name} ({size_gb:.2f} GB)")
    if len(videos) > 10:
        print(f"    ... and {len(videos) - 10} more")
else:
    print(f"\n⚠️ No .mp4 files found in {CHOLEC80_DIR}")

if CHOLECSEG_DIR.exists():
    print(f"\n✅ CholecSeg8k directory found: {CHOLECSEG_DIR}")
else:
    print(f"\n⚠️ CholecSeg8k directory not found: {CHOLECSEG_DIR}")

print("\n" + "="*60)

```

📁 Cholec80 directory: cholec80/videos
📁 CholecSeg directory: cholecseg8k_raw
📁 Model path: video01_blood_segmentation/models/best_model.pth
📁 Output directory: temporal_blood_tracking

⚙ Configuration:
video_ids: [1, 2, 3]
fps: 1
image_size: 512
batch_size: 8
threshold: 0.5
min_blood_area: 100
peak_prominence: 0.2
peak_distance: 10
smooth_window: 11

=====

PATH VERIFICATION

=====

✅ Model found: video01_blood_segmentation/models/best_model.pth

✅ Cholec80 directory found: cholec80/videos
Total videos: 80

📁 Available videos:
- video01.mp4 (0.65 GB)
- video02.mp4 (1.06 GB)
- video03.mp4 (3.81 GB)
- video04.mp4 (0.51 GB)
- video05.mp4 (0.80 GB)
- video06.mp4 (0.81 GB)
- video07.mp4 (2.00 GB)
- video08.mp4 (0.40 GB)
- video09.mp4 (1.00 GB)
- video10.mp4 (0.56 GB)
... and 70 more

✅ CholecSeg8k directory found: cholecseg8k_raw

=====

PART 1: Load Trained Model

```
In [5]: # Cell 4: Load Trained UNet++ Model
print("📦 Loading trained UNet++ model...\n")

# Create model architecture (same as training)
model = smp.UnetPlusPlus(
```

```

encoder_name="resnet50",
encoder_weights=None, # Will load trained weights
in_channels=3,
classes=1,
activation=None,
decoder_attention_type="scse"
)

# Load trained weights
checkpoint = torch.load(MODEL_PATH, map_location=device)
model.load_state_dict(checkpoint['model_state_dict'])
model = model.to(device)
model.eval()

print(f"✅ Model loaded successfully!")
print(f"\n📊 Model info:")
print(f"    Training epoch: {checkpoint.get('epoch', 'N/A')}")
print(f"    Val IoU: {checkpoint.get('val_iou', 'N/A'):.4f}")
print(f"    Val Dice: {checkpoint.get('val_dice', 'N/A'):.4f}")
print(f"\n🎯 Model ready for inference!")

```

📦 Loading trained UNet++ model...

✅ Model loaded successfully!

📊 Model info:
 Training epoch: 47
 Val IoU: 0.7891
 Val Dice: 0.8815

🎯 Model ready for inference!

In [6]: # Cell 5: Define Preprocessing Transform

```

print(f"🔧 Setting up preprocessing...\n")

# Same transform as validation (no augmentation)
transform = A.Compose([
    A.Resize(CONFIG['image_size'], CONFIG['image_size']),
    A.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ToTensorV2()
])

print(f"✅ Preprocessing pipeline ready!")
print(f"    Input size: {CONFIG['image_size']}x{CONFIG['image_size']}")
print(f"    Normalization: ImageNet statistics")

```

🔧 Setting up preprocessing...

✅ Preprocessing pipeline ready!
 Input size: 512x512
 Normalization: ImageNet statistics

PART 2: Process Videos and Generate Masks

```
In [7]: # Cell 6: Extract Frames from Videos
print("🎬 Extracting frames from Cholec80 videos...\n")

def extract_frames_from_video(video_path, fps=1):
    """
    Extract frames from video at specified FPS

    Args:
        video_path: Path to video file
        fps: Frames per second to extract

    Returns:
        frames: List of (frame_number, frame_array)
    """
    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():
        print(f"❌ Error: Cannot open video {video_path}")
        return []

    # Get video properties
    video_fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = total_frames / video_fps

    print(f"Video FPS: {video_fps:.2f}")
    print(f"Total frames: {total_frames:,}")
    print(f"Duration: {duration/60:.1f} minutes")

    # Calculate frame interval
    frame_interval = int(video_fps / fps)
    frames_to_extract = int(duration * fps)

    print(f"Extracting every {frame_interval} frames ({fps} FPS)")
    print(f"Will extract ~{frames_to_extract} frames\n")

    frames = []
    frame_count = 0

    pbar = tqdm(total=frames_to_extract, desc="Extracting frames")

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if frame_count % frame_interval == 0:
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```

        frames.append((frame_count, frame_rgb))
        pbar.update(1)

        frame_count += 1

    pbar.close()
    cap.release()

    print(f" 🟢 Extracted {len(frames)} frames\n")
    return frames

# Extract frames from all 3 videos
video_frames = {}

for video_id in CONFIG['video_ids']:
    video_path = CHOLEC80_DIR / f"video{video_id:02d}.mp4"

    print(f"{'='*70}")
    print(f"Processing Video {video_id:02d}")
    print(f"{'='*70}")

    if not video_path.exists():
        print(f"❌ Video not found: {video_path}")
        print(f"    Skipping...\n")
        continue

    frames = extract_frames_from_video(video_path, fps=CONFIG['fps'])
    video_frames[video_id] = frames

print(f"\n🟢 Frame extraction complete!")
print(f"\n📊 Summary:")
for video_id, frames in video_frames.items():
    print(f"    Video {video_id:02d}: {len(frames)} frames")

```

🎬 Extracting frames from Cholec80 videos...

=====
Processing Video 01
=====

Video FPS: 25.00
Total frames: 43,326
Duration: 28.9 minutes
Extracting every 25 frames (1 FPS)
Will extract ~1733 frames

Extracting frames: 1734it [00:12, 141.01it/s]

✓ Extracted 1734 frames

=====
Processing Video 02
=====

Video FPS: 25.00
Total frames: 70,976
Duration: 47.3 minutes
Extracting every 25 frames (1 FPS)
Will extract ~2839 frames

Extracting frames: 2840it [00:20, 140.74it/s]

✓ Extracted 2840 frames

=====
Processing Video 03
=====

Video FPS: 25.00
Total frames: 145,701
Duration: 97.1 minutes
Extracting every 25 frames (1 FPS)
Will extract ~5828 frames

Extracting frames: 5829it [00:51, 112.73it/s]

✓ Extracted 5829 frames

✓ Frame extraction complete!

 Summary:

Video 01: 1734 frames
Video 02: 2840 frames
Video 03: 5829 frames

```
In [8]: # Cell 7: Apply Model to Generate Blood Masks
print("💧 Applying UNet++ model to generate blood masks...\n")

@torch.no_grad()
def predict_blood_mask(frame, model, transform, threshold=0.5):
    """
    Predict blood mask for a single frame

    Args:
        frame: RGB frame (H, W, 3)
        model: Trained UNet++ model
        transform: Preprocessing transform
        threshold: Binary threshold

    Returns:
        mask: Binary blood mask (H, W)
        confidence: Average confidence score
    """
    # Store original size
```

```

original_h, original_w = frame.shape[:2]

# Transform
transformed = transform(image=frame)
image_tensor = transformed['image'].unsqueeze(0).to(device)

# Predict
output = model(image_tensor)
prob = torch.sigmoid(output)[0, 0].cpu().numpy()

# Binary mask
mask_resized = (prob > threshold).astype(np.uint8)

# Resize to original size
mask = cv2.resize(mask_resized, (original_w, original_h), interpolation=cv

# Calculate confidence (mean probability where blood detected)
if np.sum(mask) > 0:
    prob_resized = cv2.resize(prob, (original_w, original_h))
    confidence = np.mean(prob_resized[mask > 0])
else:
    confidence = 0.0

return mask, confidence

# Process all videos
video_masks = {}
video_confidences = {}

for video_id, frames in video_frames.items():
    print(f"{'='*70}")
    print(f"Generating masks for Video {video_id:02d}")
    print(f"{'='*70}")

    masks = []
    confidences = []

    for frame_num, frame in tqdm(frames, desc=f"Video {video_id:02d}"):
        mask, confidence = predict_blood_mask(frame, model, transform, CONFIG[
        masks.append(mask)
        confidences.append(confidence)

        # Save mask
        mask_dir = OUTPUT_DIR / "masks" / f"video{video_id:02d}"
        mask_dir.mkdir(exist_ok=True, parents=True)
        cv2.imwrite(str(mask_dir / f"frame_{frame_num:06d}.png"), mask * 255)

    video_masks[video_id] = masks
    video_confidences[video_id] = confidences

print(f"✅ Generated {len(masks)} masks")
print(f"Average confidence: {np.mean([c for c in confidences if c > 0]):")

```

🔥 Applying UNet++ model to generate blood masks...

```
In [9]: # Cell 8: Visualize BEFORE and AFTER for Sample Frames
print("🍷 Creating BEFORE/AFTER visualizations...\n")

def create_before_after_viz(frame, mask, frame_num, video_id, confidence):
    """
    Create before/after visualization with overlay
    """
    # Create overlay
```

```

overlay = frame.copy()
overlay[mask > 0] = overlay[mask > 0] * 0.5 + np.array([255, 0, 0]) * 0.5

# Calculate blood area
blood_pixels = np.sum(mask)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Before
axes[0].imshow(frame)
axes[0].set_title('BEFORE\n(Original Frame)', fontsize=14, fontweight='bold')
axes[0].axis('off')

# Mask
axes[1].imshow(mask, cmap='hot')
axes[1].set_title(f'Blood Mask\n({blood_pixels:,} pixels)', fontsize=14, fontweight='bold')
axes[1].axis('off')

# After (overlay)
axes[2].imshow(overlay.astype(np.uint8))
axes[2].set_title(f'AFTER\n(Confidence: {confidence:.2f})', fontsize=14, fontweight='bold')
axes[2].axis('off')

plt.suptitle(f'Video {video_id:02d} - Frame {frame_num}', fontsize=16, fontweight='bold')
plt.tight_layout()

return fig

# Visualize 2 sample frames from each video
for video_id in video_frames.keys():
    frames = video_frames[video_id]
    masks = video_masks[video_id]
    confidences = video_confidences[video_id]

    print(f"Video {video_id:02d}: Creating sample visualizations...")

    # Find frames with significant blood
    blood_areas = [np.sum(mask) for mask in masks]
    frames_with_blood = [(i, area) for i, area in enumerate(blood_areas) if area > 0]

    if len(frames_with_blood) == 0:
        print(f" ⚠ No significant blood detected in Video {video_id:02d}")
        continue

    # Sort by blood area and pick 2 representative frames
    frames_with_blood.sort(key=lambda x: x[1], reverse=True)
    sample_indices = [frames_with_blood[0][0]] # Highest blood
    if len(frames_with_blood) > len(frames) // 2:
        sample_indices.append(frames_with_blood[len(frames_with_blood)//2][0])

    for sample_idx in sample_indices:
        frame_num, frame = frames[sample_idx]
        mask = masks[sample_idx]

```

```

confidence = confidences[sample_idx]

fig = create_before_after_viz(frame, mask, frame_num, video_id, confid

# Save
output_path = OUTPUT_DIR / "visualizations" / f"video{video_id:02d}_fr
fig.savefig(output_path, dpi=150, bbox_inches='tight')
plt.show()
plt.close()

print(f"  ✅ Saved: {output_path.name}")

print()

print("✅ Sample visualizations complete!")

```

🤖 Creating BEFORE/AFTER visualizations...

Video 01: Creating sample visualizations...

Video 01 - Frame 28800



✅ Saved: video01_frame028800_before_after.png

Video 01 - Frame 37875



✅ Saved: video01_frame037875_before_after.png

Video 02: Creating sample visualizations...

Video 02 - Frame 57450



✅ Saved: video02_frame057450_before_after.png

Video 02 - Frame 53400



✓ Saved: video02_frame053400_before_after.png

Video 03: Creating sample visualizations...

Video 03 - Frame 102900



✓ Saved: video03_frame102900_before_after.png

Video 03 - Frame 54350



✓ Saved: video03_frame054350_before_after.png

✓ Sample visualizations complete!

PART 4: Calculate Blood Area Over Time (Industry-Grade)

```
In [10]: # Cell 9: Industry-Grade Blood Area Calculation
print("📊 Calculating blood area over time (industry-grade)...\n")

def calculate_blood_metrics(mask):
    """
    Industry-grade blood area calculation with multiple metrics
```

Handles irregular shapes using:

- Pixel counting (raw area)
- Morphological operations (noise reduction)
- Connected component analysis (multiple blood pools)
- Contour detection (irregular boundaries)

Returns:

metrics: Dictionary with multiple measurements

```
"""
# 1. Raw pixel count
raw_area = np.sum(mask > 0)

if raw_area < 10: # Too small, likely noise
    return {
        'raw_area': 0,
        'cleaned_area': 0,
        'num_regions': 0,
        'largest_region_area': 0,
        'total_perimeter': 0,
        'circularity': 0
    }

# 2. Morphological cleaning (remove small noise)
kernel = np.ones((3,3), np.uint8)
cleaned = cv2.morphologyEx(mask.astype(np.uint8), cv2.MORPH_OPEN, kernel)
cleaned = cv2.morphologyEx(cleaned, cv2.MORPH_CLOSE, kernel)
cleaned_area = np.sum(cleaned > 0)

# 3. Connected components (separate blood pools)
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(cleaned)
num_regions = num_labels - 1 # Exclude background

# 4. Find largest region
if num_regions > 0:
    areas = stats[1:, cv2.CC_STAT_AREA] # Exclude background
    largest_region_area = np.max(areas) if len(areas) > 0 else 0
else:
    largest_region_area = 0

# 5. Contour analysis (irregular boundaries)
contours, _ = cv2.findContours(cleaned, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

total_perimeter = 0
total_contour_area = 0

for contour in contours:
    perimeter = cv2.arcLength(contour, closed=True)
    area = cv2.contourArea(contour)
    total_perimeter += perimeter
    total_contour_area += area

# 6. Circularity (shape complexity measure)
# Circularity =  $4\pi \times \text{Area} / \text{Perimeter}^2$ 
```



```

# Circle = 1.0, Complex shape = closer to 0
if total_perimeter > 0:
    circularity = (4 * np.pi * total_contour_area) / (total_perimeter ** 2)
else:
    circularity = 0

return {
    'raw_area': int(raw_area),
    'cleaned_area': int(cleaned_area),
    'num_regions': int(num_regions),
    'largest_region_area': int(largest_region_area),
    'total_perimeter': float(total_perimeter),
    'circularity': float(circularity)
}

# Calculate metrics for all videos
video_blood_areas = {}
video_metrics_detailed = {}

for video_id in video_frames.keys():
    print(f"Video {video_id:02d}: Calculating blood metrics...")

    masks = video_masks[video_id]

    blood_areas = []
    metrics_list = []

    for mask in tqdm(masks, desc=f"Video {video_id:02d}", leave=False):
        metrics = calculate_blood_metrics(mask)
        blood_areas.append(metrics['cleaned_area']) # Use cleaned area
        metrics_list.append(metrics)

    video_blood_areas[video_id] = blood_areas
    video_metrics_detailed[video_id] = metrics_list

    print(f"    ✓ Calculated {len(blood_areas)} measurements")
    print(f"    Mean blood area: {np.mean([a for a in blood_areas if a > 0]):,.0f} pixels")
    print(f"    Max blood area: {np.max(blood_areas):,.0f} pixels")
    print(f"    Frames with blood: {len([a for a in blood_areas if a > CONFIG['min_area']])}")
    print()

print("✓ Blood area calculation complete!")

```

🚧 Calculating blood area over time (industry-grade)...

Video 01: Calculating blood metrics...

```

✓ Calculated 1734 measurements
Mean blood area: 9,795 pixels
Max blood area: 39,148 pixels
Frames with blood: 1294

```

Video 02: Calculating blood metrics...

✓ Calculated 2840 measurements
Mean blood area: 6,565 pixels
Max blood area: 43,350 pixels
Frames with blood: 1938

Video 03: Calculating blood metrics...

✓ Calculated 5829 measurements
Mean blood area: 10,265 pixels
Max blood area: 71,814 pixels
Frames with blood: 5375

✓ Blood area calculation complete!

PART 5: Plot Blood Area vs Time (All 3 Videos)

```
In [11]: # Cell 10: Smooth Curves and Plot Blood Area Over Time
print("✓ Plotting blood area over time...\n")

def smooth_curve(data, window=11, poly=2):
    """
    Smooth curve using Savitzky-Golay filter
    """
    if len(data) < window:
        return np.array(data)
    return savgol_filter(data, window_length=window, polyorder=poly)

# Prepare data
video_blood_areas_smooth = {}

for video_id, blood_areas in video_blood_areas.items():
    # Smooth the curve
    smoothed = smooth_curve(blood_areas, window=CONFIG['smooth_window'])
    video_blood_areas_smooth[video_id] = smoothed

# Create comprehensive plot
fig, axes = plt.subplots(2, 2, figsize=(20, 12))

colors = ['#e74c3c', '#3498db', '#2ecc71'] # Red, Blue, Green

# Plot 1: All videos together (raw)
ax = axes[0, 0]
for i, video_id in enumerate(video_blood_areas_smooth.keys()):
    blood_areas = video_blood_areas_smooth[video_id]
    time_points = np.arange(len(blood_areas)) / CONFIG['fps'] / 60 # Convert

    ax.plot(time_points, blood_areas, color=colors[i], alpha=0.3, linewidth=0.
```

```

    ax.plot(time_points, video_blood_areas_smooth[video_id], color=colors[i],
            linewidth=2, label=f'Video {video_id:02d}')

ax.set_xlabel('Time (minutes)', fontsize=12)
ax.set_ylabel('Blood Area (pixels)', fontsize=12)
ax.set_title('Blood Area Over Time - All Videos\n(Solid = Smoothed, Faint = Raw)',
            fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

# Plot 2: Individual videos (smoothed only)
ax = axes[0, 1]
for i, video_id in enumerate(video_blood_areas.keys()):
    smoothed = video_blood_areas_smooth[video_id]
    time_points = np.arange(len(smoothed)) / CONFIG['fps'] / 60

    ax.plot(time_points, smoothed, color=colors[i], linewidth=3,
            label=f'Video {video_id:02d}', marker='o', markersize=2, markevery=10)

ax.set_xlabel('Time (minutes)', fontsize=12)
ax.set_ylabel('Blood Area (pixels)', fontsize=12)
ax.set_title('Smoothed Blood Area Curves', fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

# Plot 3: Cumulative blood loss
ax = axes[1, 0]
for i, video_id in enumerate(video_blood_areas.keys()):
    smoothed = video_blood_areas_smooth[video_id]
    cumulative = np.cumsum(smoothed)
    time_points = np.arange(len(smoothed)) / CONFIG['fps'] / 60

    ax.plot(time_points, cumulative, color=colors[i], linewidth=3,
            label=f'Video {video_id:02d}')

ax.set_xlabel('Time (minutes)', fontsize=12)
ax.set_ylabel('Cumulative Blood (pixel·frames)', fontsize=12)
ax.set_title('Cumulative Blood Loss Over Time', fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)

# Plot 4: Statistics comparison
ax = axes[1, 1]
video_ids_list = list(video_blood_areas.keys())
x_pos = np.arange(len(video_ids_list))

# Mean blood area
means = [np.mean([a for a in video_blood_areas[vid] if a > 0]) for vid in video_ids_list]
# Max blood area
maxs = [np.max(video_blood_areas[vid]) for vid in video_ids_list]
# Total blood
totals = [np.sum(video_blood_areas[vid]) for vid in video_ids_list]

```

```

width = 0.25
ax.bar(x_pos - width, means, width, label='Mean Area', color='#3498db')
ax.bar(x_pos, maxs, width, label='Max Area', color='#e74c3c')
ax.bar(x_pos + width, np.array(totals)/1000, width, label='Total (x1000)', col

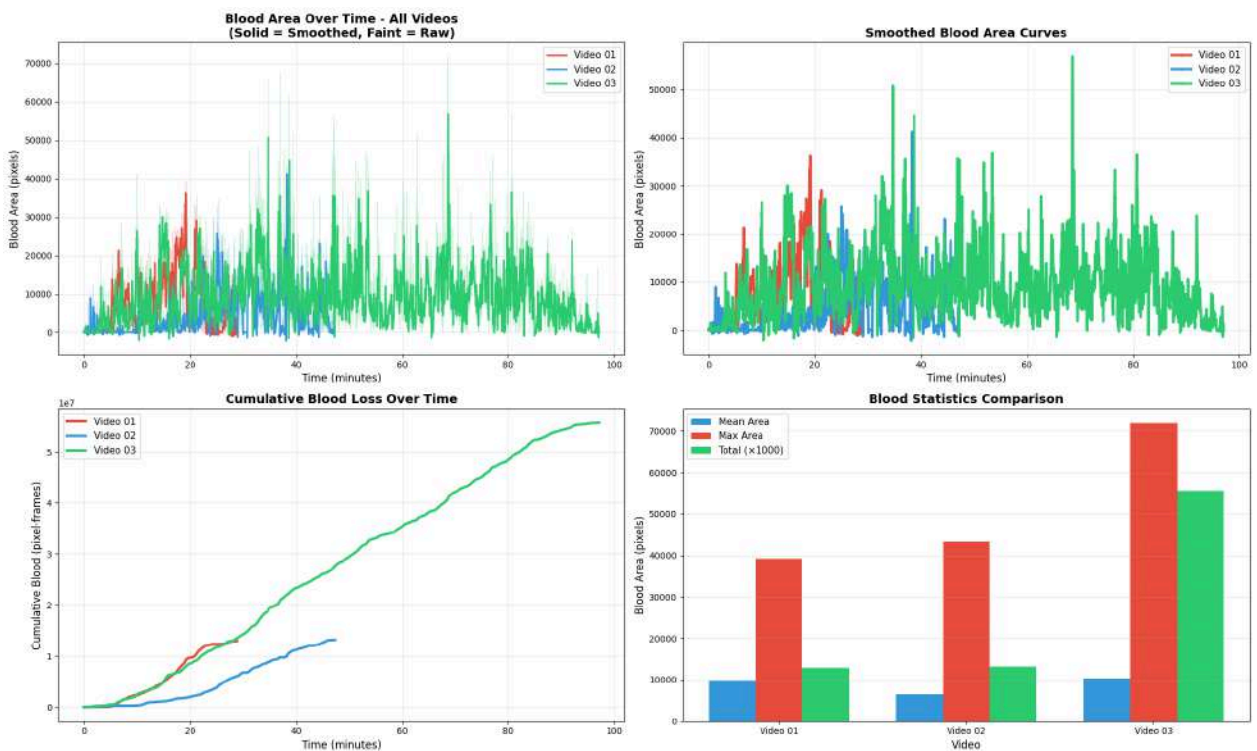
ax.set_xlabel('Video', fontsize=12)
ax.set_ylabel('Blood Area (pixels)', fontsize=12)
ax.set_title('Blood Statistics Comparison', fontsize=14, fontweight='bold')
ax.set_xticks(x_pos)
ax.set_xticklabels([f'Video {vid:02d}' for vid in video_ids_list])
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'visualizations/blood_area_temporal_analysis.png', dp
plt.show()

print("✅ Temporal plots saved!")
print(f"    File: {OUTPUT_DIR / 'visualizations/blood_area_temporal_analysis.pr

```

📈 Plotting blood area over time...



✅ Temporal plots saved!
 File: temporal_blood_tracking/visualizations/blood_area_temporal_analysis.png



PART 6: Peak Detection (Maximum Bleeding Events)

```
In [12]: # Cell 11: Detect Peaks (Bleeding Events)
print("📌 Detecting peaks (bleeding events)...\n")

def detect_bleeding_events(blood_areas_smooth, prominence=0.2, distance=10):
    """
    Detect bleeding events using peak detection

    Args:
        blood_areas_smooth: Smoothed blood area time series
        prominence: Minimum peak prominence (relative to baseline)
        distance: Minimum frames between peaks

    Returns:
        peaks: Peak indices
        properties: Peak properties (height, prominence, width)
    """
    # Normalize data for prominence calculation
    max_area = np.max(blood_areas_smooth)
    if max_area == 0:
        return np.array([]), {}

    normalized = blood_areas_smooth / max_area

    # Find peaks
    peaks, properties = find_peaks(
        normalized,
        prominence=prominence,
        distance=distance,
        width=2, # Minimum width
        height=0.1 # Minimum height (10% of max)
    )

    # Convert properties back to original scale
    if len(peaks) > 0:
        properties['peak_heights'] = blood_areas_smooth[peaks]
        properties['prominences'] = properties['prominences'] * max_area

    return peaks, properties

# Detect peaks for all videos
video_peaks = {}
video_peak_properties = {}

for video_id in video_blood_areas.keys():
    smoothed = video_blood_areas_smooth[video_id]

    peaks, properties = detect_bleeding_events(
```

```
        smoothed,
        prominence=CONFIG['peak_prominence'],
        distance=CONFIG['peak_distance']
    )

    video_peaks[video_id] = peaks
    video_peak_properties[video_id] = properties

    print(f"Video {video_id:02d}:")
    print(f"    Detected {len(peaks)} bleeding events")

    if len(peaks) > 0:
        print(f"    Peak frames: {peaks.tolist()}")
        print(f"    Peak heights: {properties['peak_heights'].astype(int).tolist}")
        print(f"    Prominences: {properties['prominences'].astype(int).tolist()}")
        print()

    print("✅ Peak detection complete!")
```

Detecting peaks (bleeding events)...

Video 01:

Detected 27 bleeding events

Peak frames: [317, 398, 432, 505, 547, 642, 697, 713, 746, 806, 846, 915, 960, 1002, 1032, 1049, 1086, 1107, 1151, 1173, 1260, 1275, 1333, 1369, 1435, 1633, 1676]

Peak heights: [13810, 21331, 15467, 12718, 9481, 15458, 12089, 12349, 14427, 18102, 13089, 18334, 22122, 18041, 22840, 24602, 24453, 27281, 36326, 17718, 26978, 29136, 18513, 18455, 13449, 14772, 11143]

Prominences: [13405, 21548, 11143, 10427, 8524, 13649, 9145, 9861, 10536, 15562, 9894, 11847, 17076, 11759, 14985, 8034, 7657, 18219, 36676, 15916, 10545, 28690, 8912, 10933, 13924, 15485, 12143]

Video 02:

Detected 34 bleeding events

Peak frames: [76, 704, 968, 1024, 1216, 1281, 1348, 1405, 1470, 1505, 1527, 1565, 1606, 1640, 1689, 1719, 1769, 1883, 1936, 1971, 2070, 2103, 2138, 2199, 2212, 2280, 2299, 2347, 2357, 2403, 2463, 2532, 2669, 2738]

Peak heights: [8950, 9994, 8864, 10100, 13224, 12081, 19894, 15202, 17275, 25673, 20634, 20895, 13597, 11966, 12264, 11846, 20282, 18709, 10963, 14228, 18531, 16467, 12804, 14166, 20238, 24058, 41207, 16073, 15338, 12000, 17200, 15594, 23108, 18587]

Prominences: [9646, 10709, 9138, 10309, 13390, 11531, 20840, 12044, 17969, 26765, 8441, 18226, 9499, 8680, 10056, 10877, 21541, 19511, 9994, 14448, 18963, 16736, 11980, 9071, 21264, 25460, 42483, 11219, 8988, 11682, 17935, 14846, 23743, 16215]

Video 03:

Detected 85 bleeding events

Peak frames: [192, 361, 435, 547, 601, 652, 692, 752, 859, 890, 1003, 1160, 1239, 1287, 1315, 1375, 1469, 1598, 1626, 1767, 1793, 1847, 1888, 1946, 1965, 1983, 2086, 2205, 2221, 2259, 2324, 2368, 2420, 2463, 2578, 2614, 2664, 2799, 2817, 2839, 2868, 2931, 2988, 3011, 3069, 3111, 3130, 3148, 3212, 3334, 3518, 3621, 3698, 3762, 3878, 4002, 4037, 4058, 4069, 4099, 4118, 4134, 4237, 4270, 4301, 4403, 4503, 4526, 4597, 4664, 4689, 4751, 4791, 4811, 4841, 4874, 4894, 4937, 4965, 5009, 5060, 5081, 5247, 5262, 5521]

Peak heights: [11836, 13659, 16752, 13311, 26512, 15318, 13194, 16510, 28016, 30070, 11881, 21664, 16666, 25859, 27218, 16774, 21160, 18554, 17612, 20891, 17261, 20614, 28085, 26812, 32037, 30800, 50797, 31404, 35547, 22029, 44679, 25458, 15000, 15798, 19263, 13869, 20464, 22252, 35664, 35361, 27755, 18018, 24673, 22831, 21434, 34795, 28563, 22338, 36881, 19854, 20219, 25039, 12439, 27794, 19730, 22139, 20591, 22447, 22851, 23224, 56919, 33247, 22081, 16001, 13457, 22356, 19125, 20966, 33299, 19834, 14356, 18785, 25950, 21853, 36582, 19699, 17082, 21610, 21500, 23649, 21942, 16979, 20553, 13090, 23801]

Prominences: [12045, 11863, 16658, 11843, 27091, 16340, 13227, 17162, 26299, 31738, 12247, 20227, 13750, 15848, 27842, 12066, 18970, 17213, 11470, 15693, 11441, 11390, 29123, 16965, 31644, 12806, 52334, 14627, 34389, 15037, 46039, 20071, 12116, 13717, 17990, 11944, 18874, 15114, 35769, 33076, 21571, 14906, 21616, 17857, 15797, 32133, 19561, 12148, 38117, 18353, 15361, 24431, 11724, 27574, 17366, 16955, 15353, 18140, 15357, 14681, 58256, 14435, 20211, 11686, 11430, 20637, 11634, 17821, 33348, 19936, 13772, 18592, 23769, 16159, 36688, 16083, 12815, 18954, 13544, 24590, 16848, 13516, 21059, 11466, 24792]

✓ Peak detection complete!

```
In [13]: # Cell 12: Visualize Peaks on Blood Curves
print("📊 Visualizing peaks on blood curves...\n")

fig, axes = plt.subplots(len(video_blood_areas), 1, figsize=(16, 5*len(video_blood_areas)))

if len(video_blood_areas) == 1:
    axes = [axes]

for i, video_id in enumerate(video_blood_areas.keys()):
    ax = axes[i]

    smoothed = video_blood_areas_smooth[video_id]
    peaks = video_peaks[video_id]
    time_points = np.arange(len(smoothed)) / CONFIG['fps'] / 60

    # Plot curve
    ax.plot(time_points, smoothed, color=colors[i], linewidth=3, label='Blood Area')

    # Mark peaks
    if len(peaks) > 0:
        ax.plot(time_points[peaks], smoothed[peaks], 'r*', markersize=20,
                label=f'Bleeding Events (n={len(peaks)})', zorder=5)

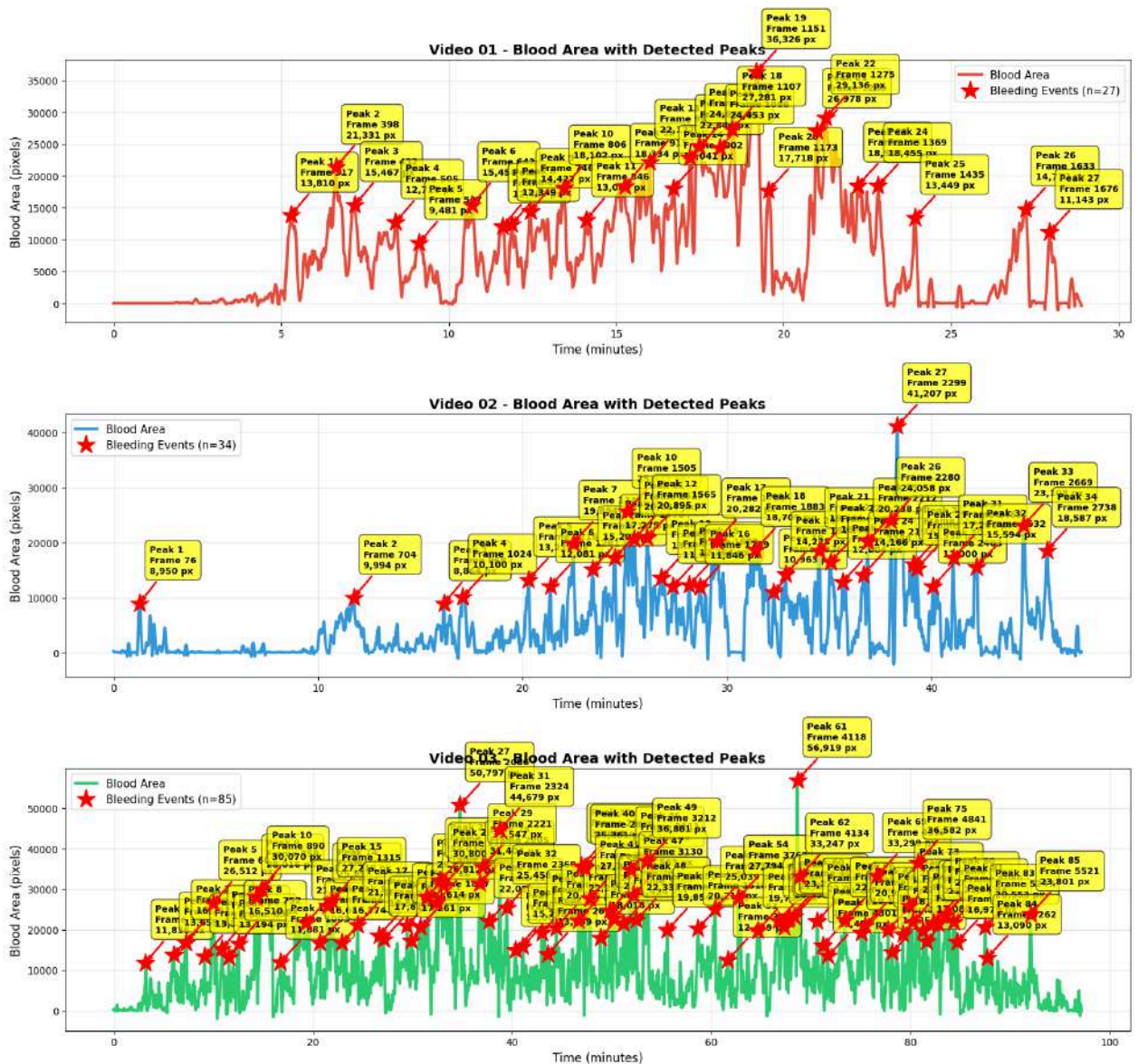
    # Annotate peaks
    for j, peak_idx in enumerate(peaks):
        ax.annotate(f'Peak {j+1}\nFrame {peak_idx}\n{int(smoothed[peak_idx])}',
                    xy=(time_points[peak_idx], smoothed[peak_idx]),
                    xytext=(10, 30), textcoords='offset points',
                    bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5),
                    arrowprops=dict(arrowstyle='->', connectionstyle='arc3,rad=40',
                                    color='red', lw=2),
                    fontsize=9, fontweight='bold')

    ax.set_xlabel('Time (minutes)', fontsize=12)
    ax.set_ylabel('Blood Area (pixels)', fontsize=12)
    ax.set_title(f'Video {video_id:02d} - Blood Area with Detected Peaks',
                fontsize=14, fontweight='bold')
    ax.legend(fontsize=11)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'visualizations/blood_curves_with_peaks.png', dpi=300)
plt.show()

print("✓ Peak visualization saved!")
print(f"    File: {OUTPUT_DIR / 'visualizations/blood_curves_with_peaks.png'}")
```

📊 Visualizing peaks on blood curves...



✓ Peak visualization saved!
 File: temporal_blood_tracking/visualizations/blood_curves_with_peaks.png

🎯 PART 7: Show Peak Frames (Maximum Bleeding Moments)

```
In [ ]: # Cell 13: Visualize Peak Frames (Before and After)
print("🎯 Visualizing peak frames (maximum bleeding moments)...\n")

def visualize_peak_frame(video_id, peak_idx, peak_num, frames, masks, blood_ar
    """
    Create comprehensive visualization for a peak frame
    """
    frame_num, frame = frames[peak_idx]
```



```

mask = masks[peak_idx]

# Create overlay
overlay = frame.copy()
overlay[mask > 0] = overlay[mask > 0] * 0.4 + np.array([255, 0, 0]) * 0.6

# Create figure
fig = plt.figure(figsize=(20, 10))
gs = fig.add_gridspec(2, 3, hspace=0.3, wspace=0.3)

# Top row: Before, Mask, After
ax1 = fig.add_subplot(gs[0, 0])
ax1.imshow(frame)
ax1.set_title('BEFORE\n(Original Frame)', fontsize=14, fontweight='bold')
ax1.axis('off')

ax2 = fig.add_subplot(gs[0, 1])
ax2.imshow(mask, cmap='hot')
ax2.set_title('Blood Mask\n({blood_area:,} pixels)', fontsize=14, fontwei
ax2.axis('off')

ax3 = fig.add_subplot(gs[0, 2])
ax3.imshow(overlay.astype(np.uint8))
ax3.set_title('AFTER\n(With Detection)', fontsize=14, fontweight='bold')
ax3.axis('off')

# Bottom row: Context (frames before and after peak)
context_indices = [max(0, peak_idx-1), peak_idx, min(len(frames)-1, peak_i
context_titles = ['Before Peak', 'PEAK', 'After Peak']

for i, (ctx_idx, title) in enumerate(zip(context_indices, context_titles))
    ax = fig.add_subplot(gs[1, i])
    ctx_frame = frames[ctx_idx][1]
    ctx_mask = masks[ctx_idx]
    ctx_overlay = ctx_frame.copy()
    ctx_overlay[ctx_mask > 0] = ctx_overlay[ctx_mask > 0] * 0.5 + np.array

    ax.imshow(ctx_overlay.astype(np.uint8))
    ax.set_title(f'{title}\nFrame {frames[ctx_idx][0]}', fontsize=12, font
    ax.axis('off')

    if i == 1: # Peak frame
        for spine in ax.spines.values():
            spine.set_edgecolor('red')
            spine.set_linewidth(5)

fig.suptitle(f'Video {video_id:02d} - Peak {peak_num} at Frame {frame_num}
            fontsize=16, fontweight='bold', y=0.98)

return fig

# Visualize all peaks
for video_id in video_frames.keys():

```

```

peaks = video_peaks[video_id]

if len(peaks) == 0:
    print(f"Video {video_id:02d}: No peaks detected\n")
    continue

print(f"Video {video_id:02d}: Visualizing {len(peaks)} peaks...")

frames = video_frames[video_id]
masks = video_masks[video_id]
smoothed = video_blood_areas_smooth[video_id]

for peak_num, peak_idx in enumerate(peaks, 1):
    blood_area = int(smoothed[peak_idx])

    fig = visualize_peak_frame(video_id, peak_idx, peak_num, frames, masks)

    # Save
    output_path = OUTPUT_DIR / "visualizations" / f"video{video_id:02d}_pe{peak_num:02d}.png"
    fig.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.show()
    plt.close()

    print(f"✅ Peak {peak_num}: Frame {frames[peak_idx][0]} ({blood_area} pixels)")

print()

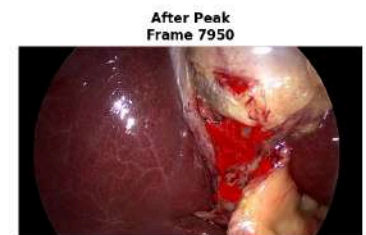
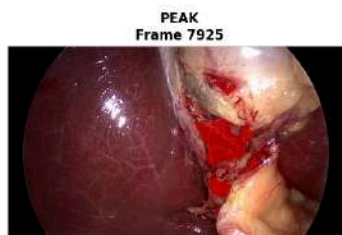
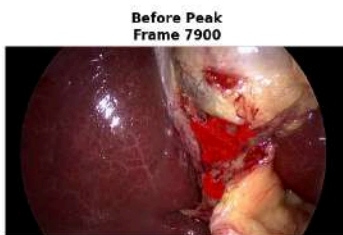
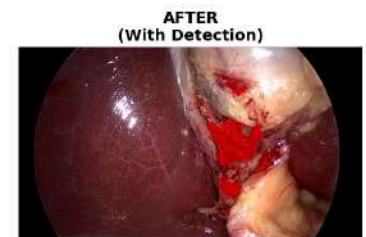
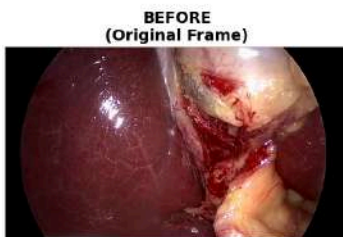
print("✅ Peak frame visualizations complete!")

```

🎯 Visualizing peak frames (maximum bleeding moments)...

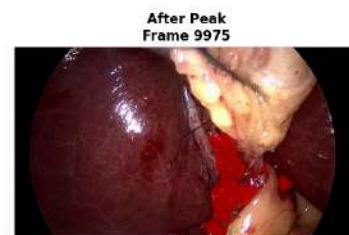
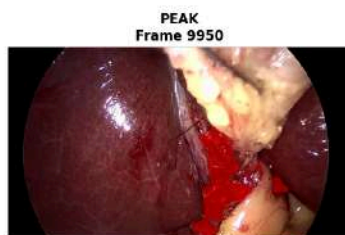
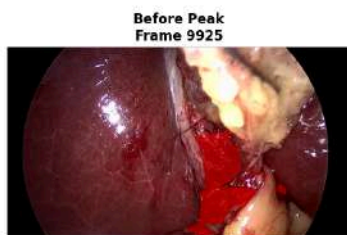
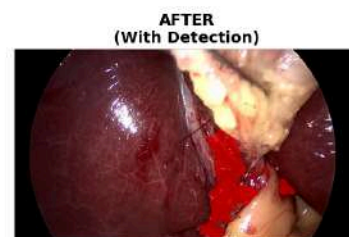
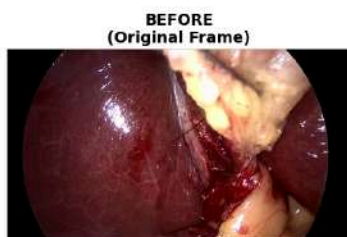
Video 01: Visualizing 27 peaks...

Video 01 - Peak 1 at Frame 7925
Maximum Bleeding Event



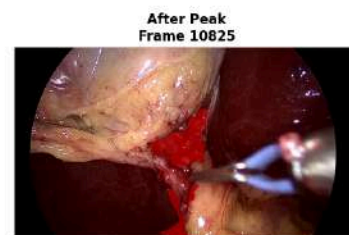
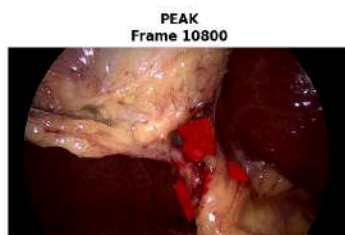
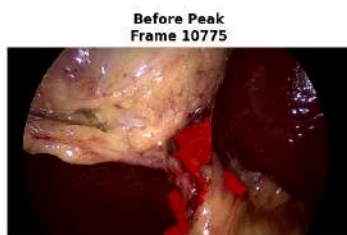
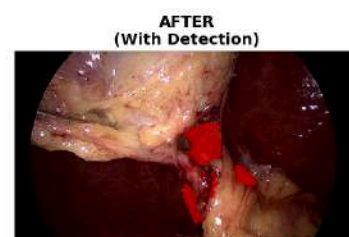
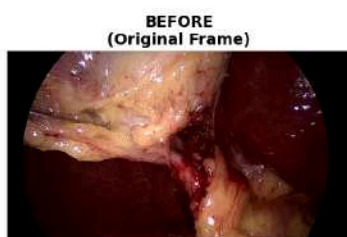
✅ Peak 1: Frame 7925 (13,810 pixels)

**Video 01 - Peak 2 at Frame 9950
Maximum Bleeding Event**



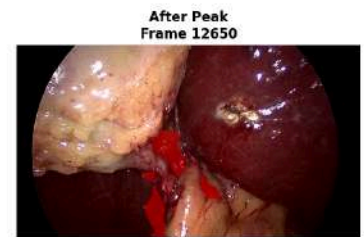
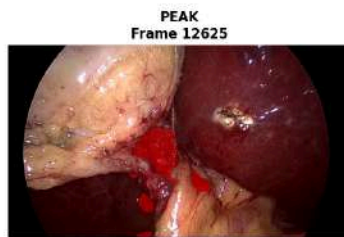
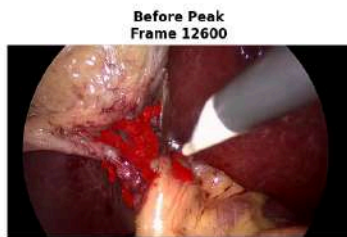
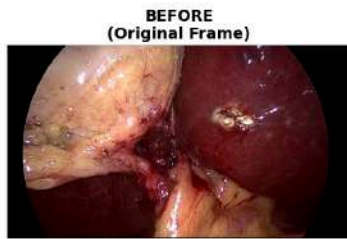
✓ Peak 2: Frame 9950 (21,331 pixels)

**Video 01 - Peak 3 at Frame 10800
Maximum Bleeding Event**



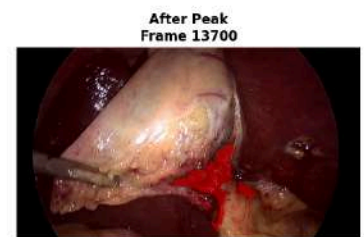
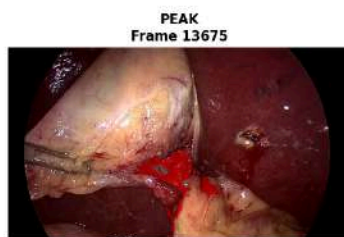
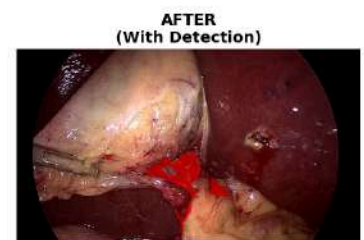
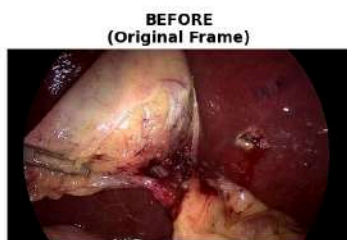
✓ Peak 3: Frame 10800 (15,467 pixels)

Video 01 - Peak 4 at Frame 12625
Maximum Bleeding Event



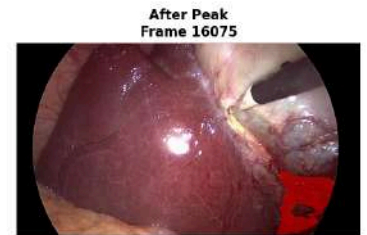
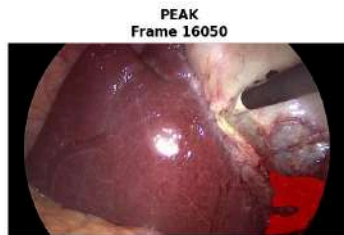
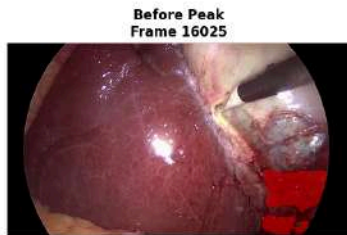
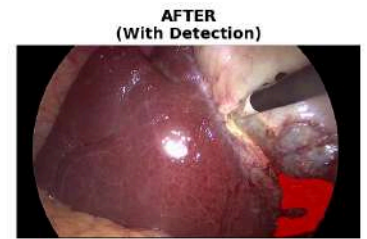
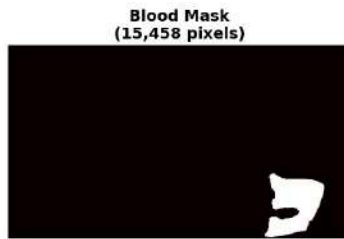
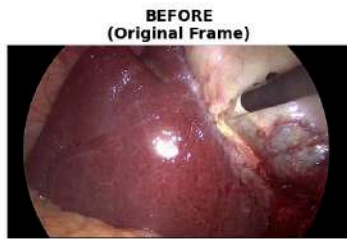
✓ Peak 4: Frame 12625 (12,718 pixels)

Video 01 - Peak 5 at Frame 13675
Maximum Bleeding Event



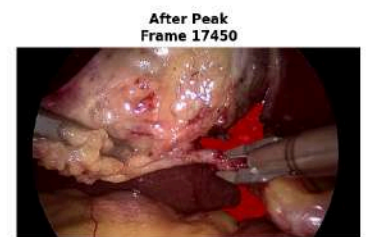
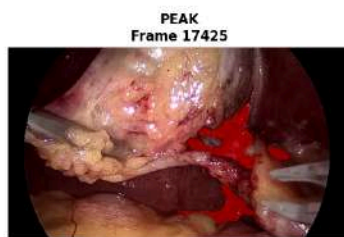
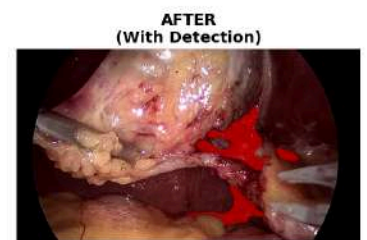
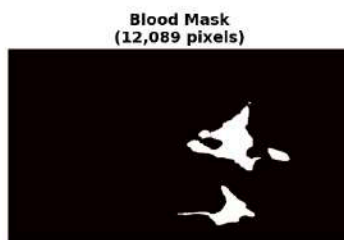
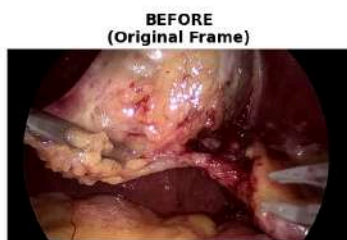
✓ Peak 5: Frame 13675 (9,481 pixels)

**Video 01 - Peak 6 at Frame 16050
Maximum Bleeding Event**



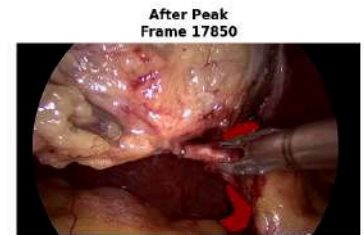
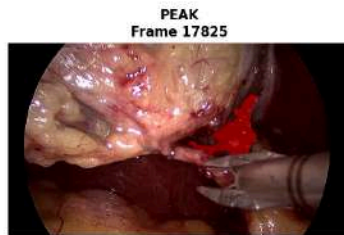
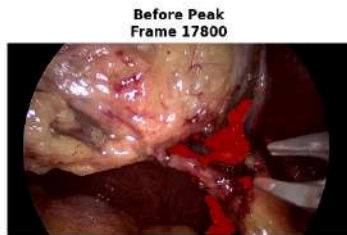
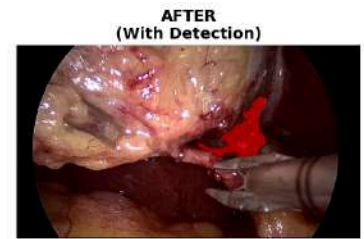
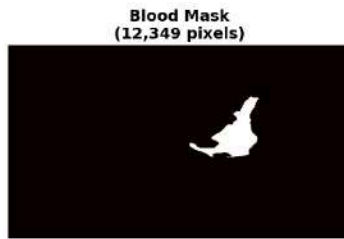
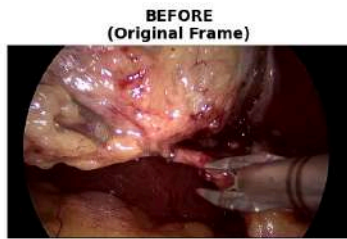
✓ Peak 6: Frame 16050 (15,458 pixels)

**Video 01 - Peak 7 at Frame 17425
Maximum Bleeding Event**



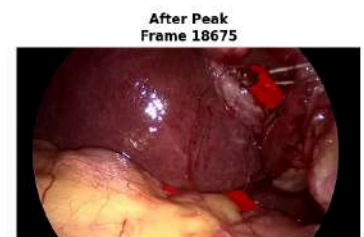
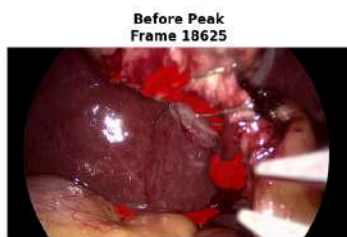
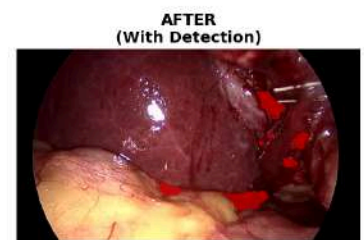
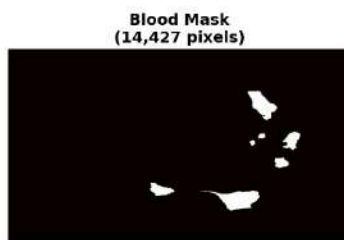
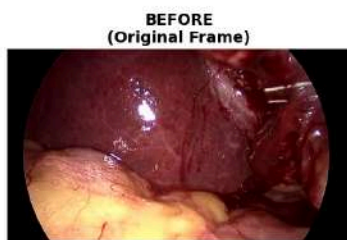
✓ Peak 7: Frame 17425 (12,089 pixels)

**Video 01 - Peak 8 at Frame 17825
Maximum Bleeding Event**



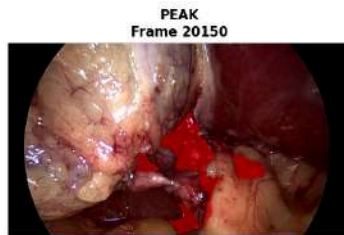
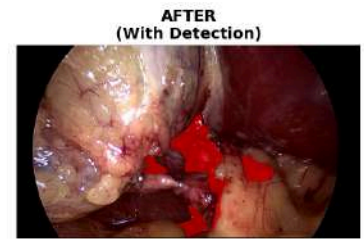
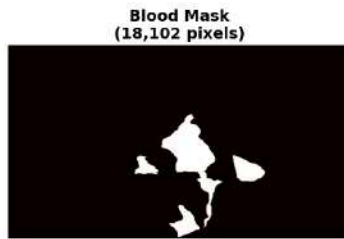
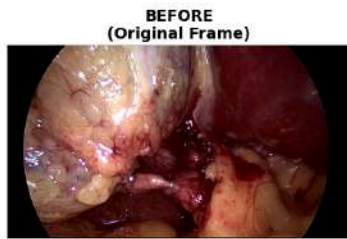
✓ Peak 8: Frame 17825 (12,349 pixels)

**Video 01 - Peak 9 at Frame 18650
Maximum Bleeding Event**



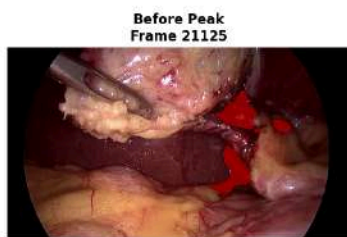
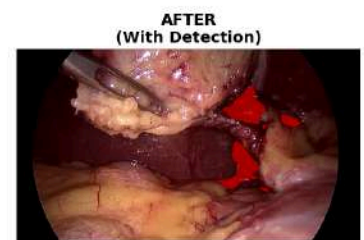
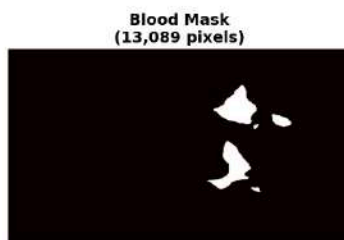
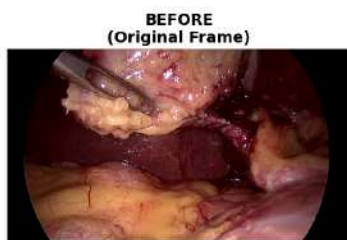
✓ Peak 9: Frame 18650 (14,427 pixels)

**Video 01 - Peak 10 at Frame 20150
Maximum Bleeding Event**



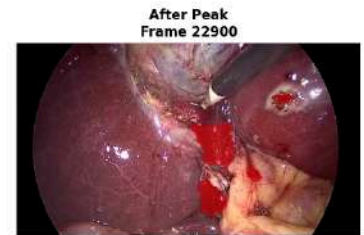
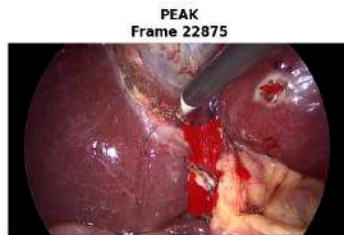
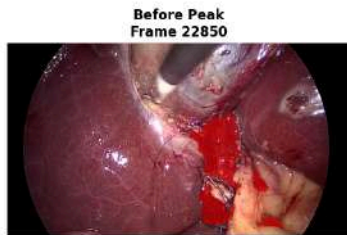
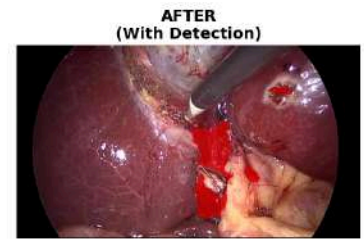
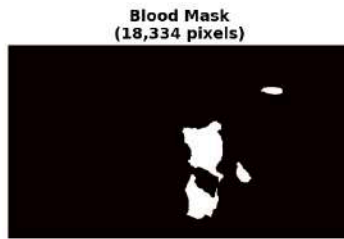
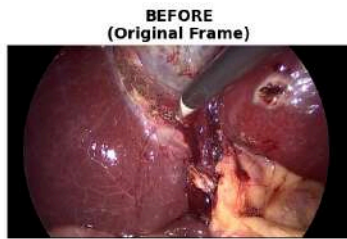
✓ Peak 10: Frame 20150 (18,102 pixels)

**Video 01 - Peak 11 at Frame 21150
Maximum Bleeding Event**



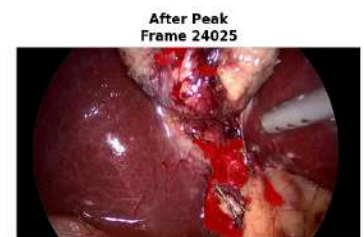
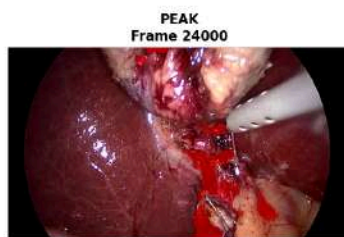
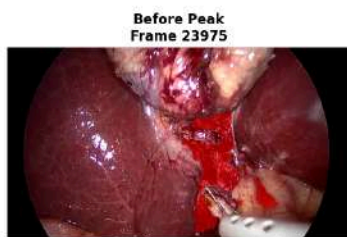
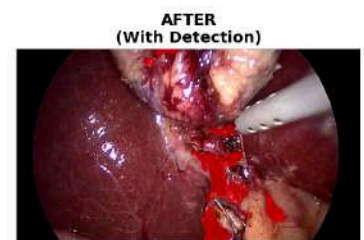
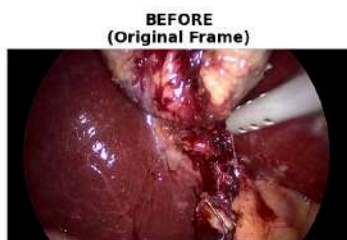
✓ Peak 11: Frame 21150 (13,089 pixels)

**Video 01 - Peak 12 at Frame 22875
Maximum Bleeding Event**



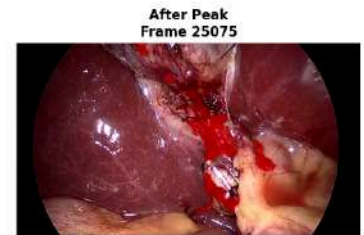
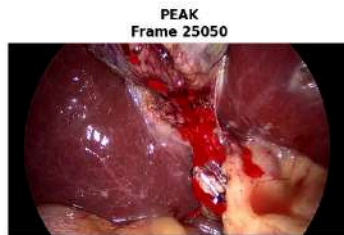
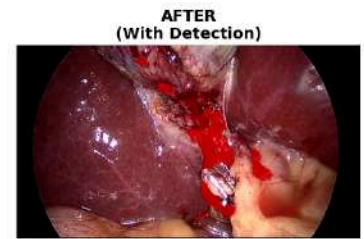
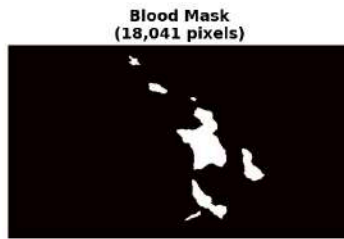
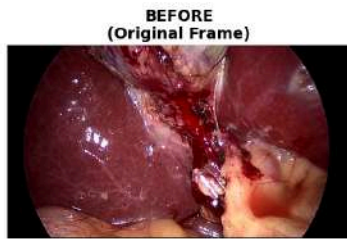
✓ Peak 12: Frame 22875 (18,334 pixels)

**Video 01 - Peak 13 at Frame 24000
Maximum Bleeding Event**



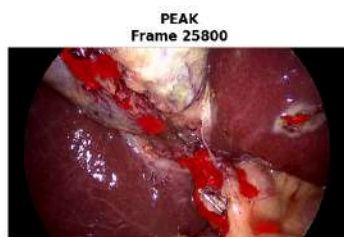
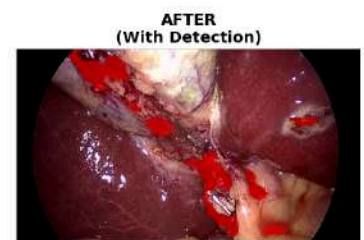
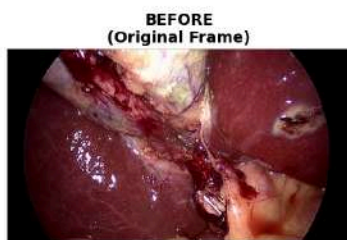
✓ Peak 13: Frame 24000 (22,122 pixels)

**Video 01 - Peak 14 at Frame 25050
Maximum Bleeding Event**



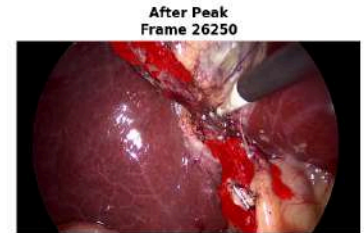
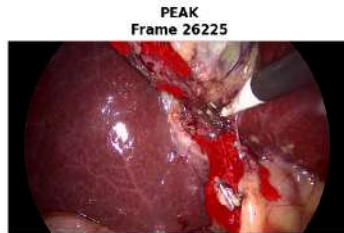
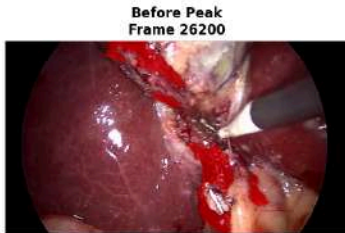
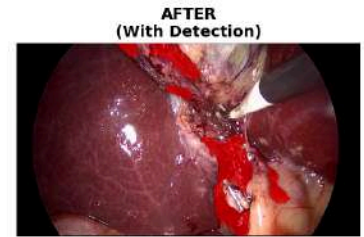
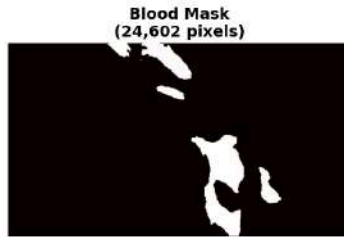
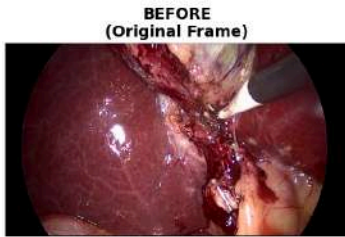
✓ Peak 14: Frame 25050 (18,041 pixels)

**Video 01 - Peak 15 at Frame 25800
Maximum Bleeding Event**



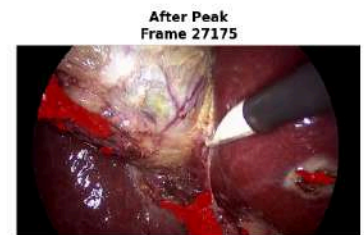
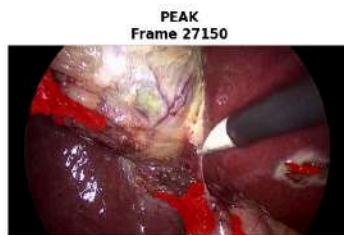
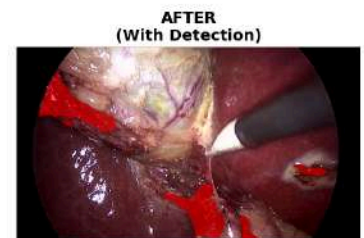
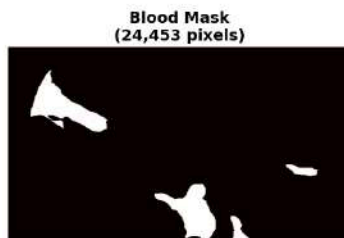
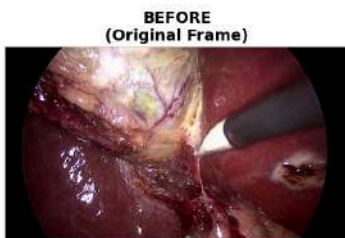
✓ Peak 15: Frame 25800 (22,840 pixels)

**Video 01 - Peak 16 at Frame 26225
Maximum Bleeding Event**



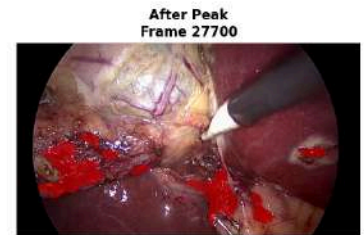
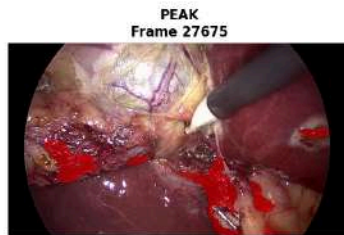
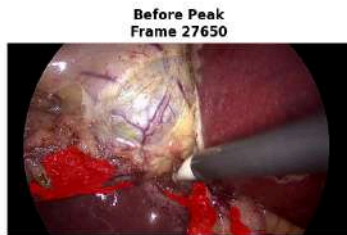
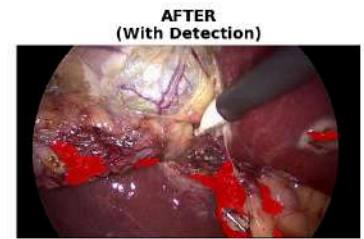
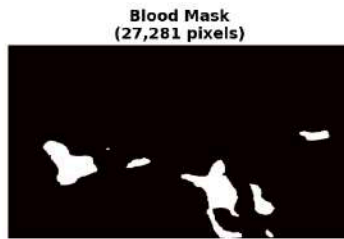
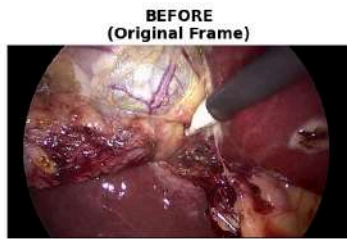
✓ Peak 16: Frame 26225 (24,602 pixels)

**Video 01 - Peak 17 at Frame 27150
Maximum Bleeding Event**



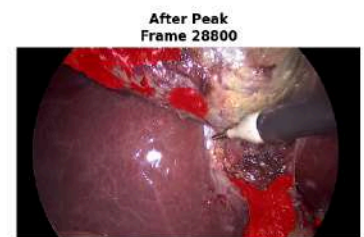
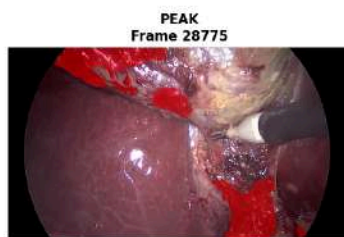
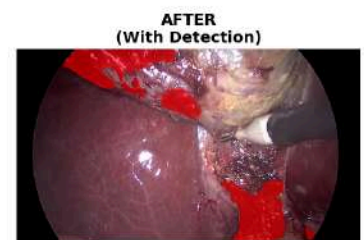
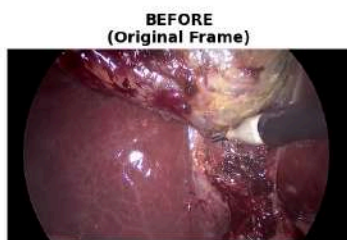
✓ Peak 17: Frame 27150 (24,453 pixels)

**Video 01 - Peak 18 at Frame 27675
Maximum Bleeding Event**



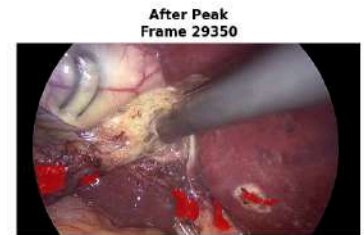
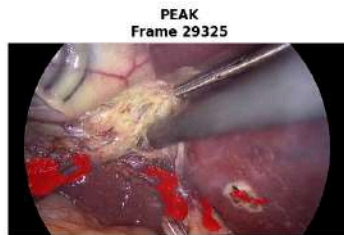
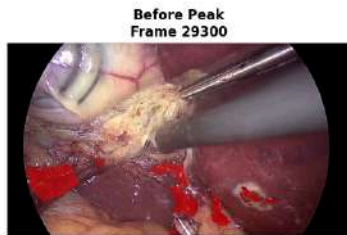
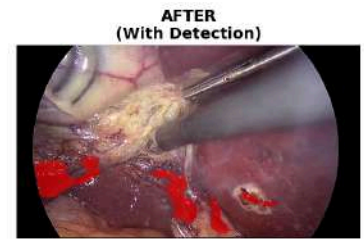
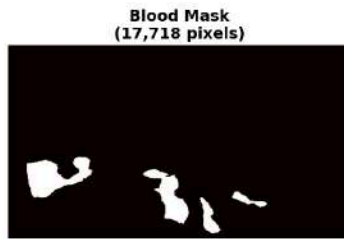
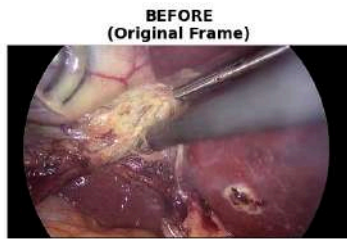
✓ Peak 18: Frame 27675 (27,281 pixels)

**Video 01 - Peak 19 at Frame 28775
Maximum Bleeding Event**



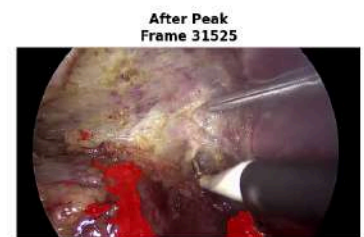
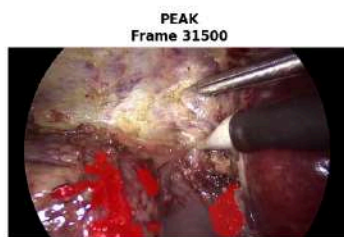
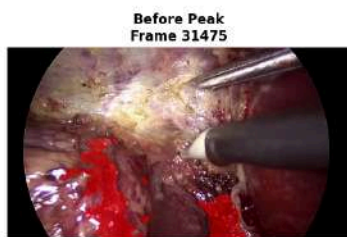
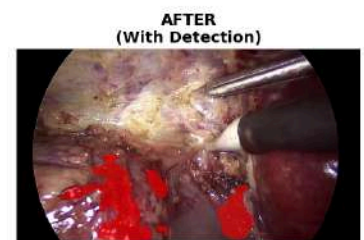
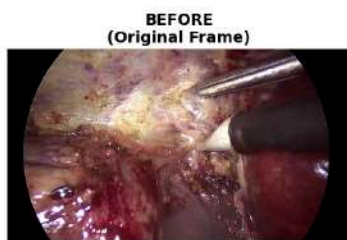
✓ Peak 19: Frame 28775 (36,326 pixels)

Video 01 - Peak 20 at Frame 29325
Maximum Bleeding Event



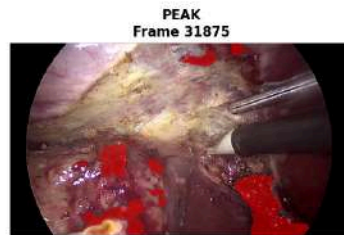
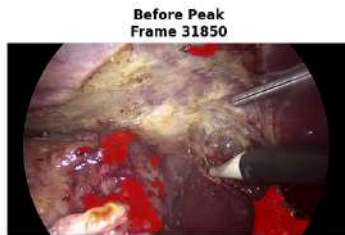
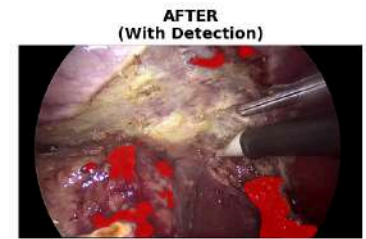
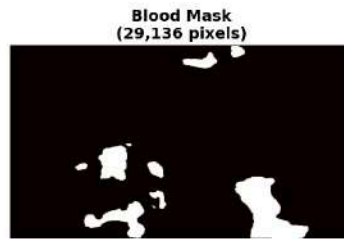
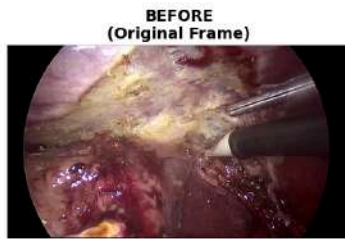
✓ Peak 20: Frame 29325 (17,718 pixels)

Video 01 - Peak 21 at Frame 31500
Maximum Bleeding Event



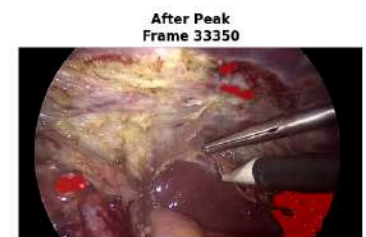
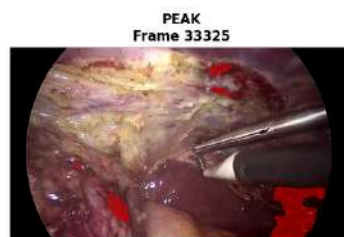
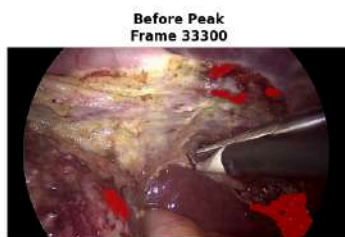
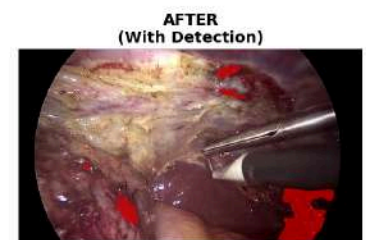
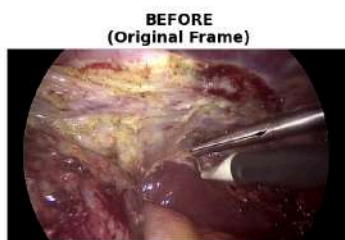
✓ Peak 21: Frame 31500 (26,978 pixels)

**Video 01 - Peak 22 at Frame 31875
Maximum Bleeding Event**



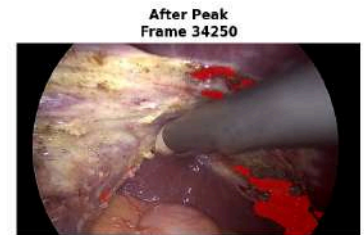
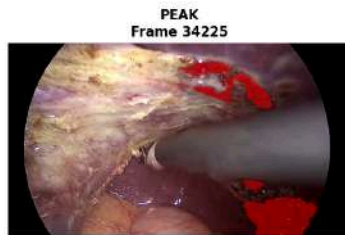
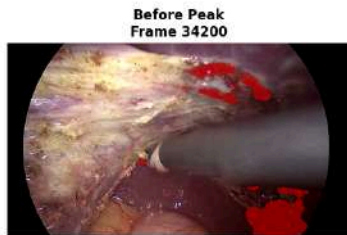
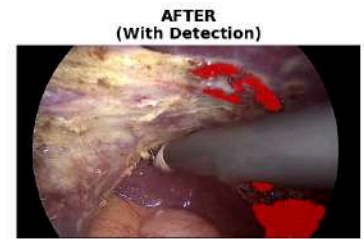
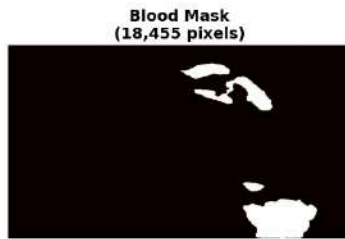
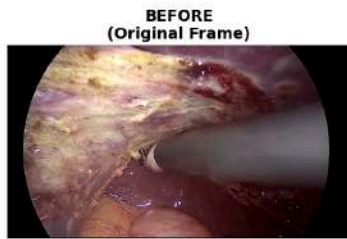
✓ Peak 22: Frame 31875 (29,136 pixels)

**Video 01 - Peak 23 at Frame 33325
Maximum Bleeding Event**



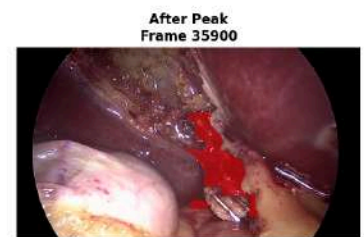
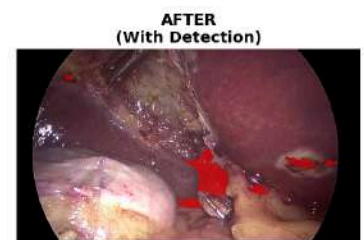
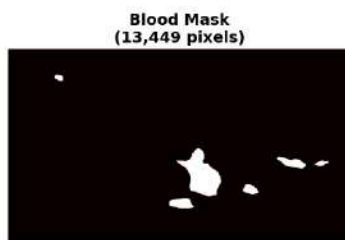
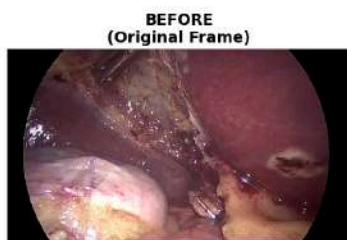
✓ Peak 23: Frame 33325 (18,513 pixels)

**Video 01 - Peak 24 at Frame 34225
Maximum Bleeding Event**



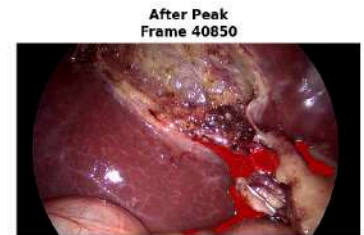
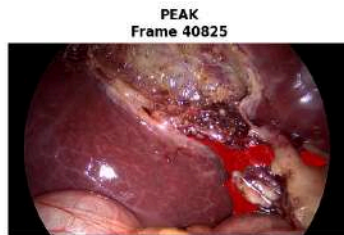
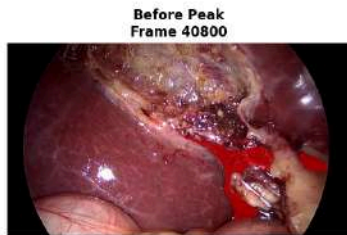
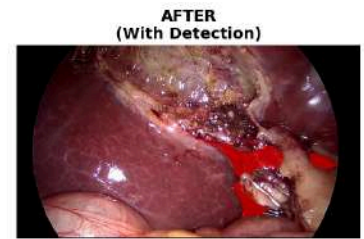
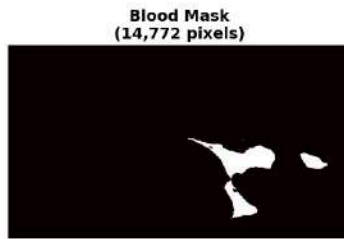
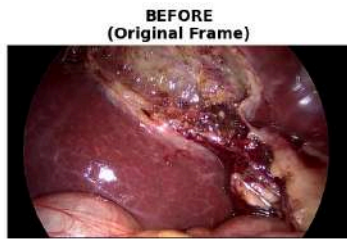
✓ Peak 24: Frame 34225 (18,455 pixels)

**Video 01 - Peak 25 at Frame 35875
Maximum Bleeding Event**



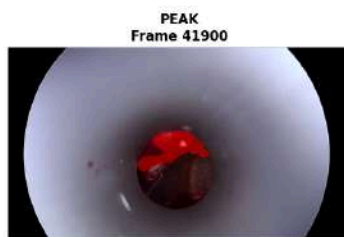
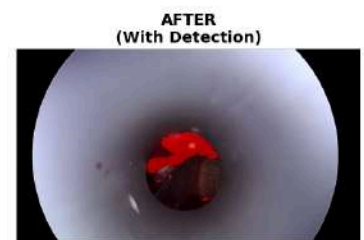
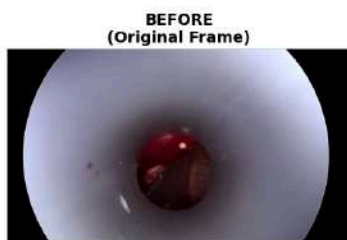
✓ Peak 25: Frame 35875 (13,449 pixels)

**Video 01 - Peak 26 at Frame 40825
Maximum Bleeding Event**



✓ Peak 26: Frame 40825 (14,772 pixels)

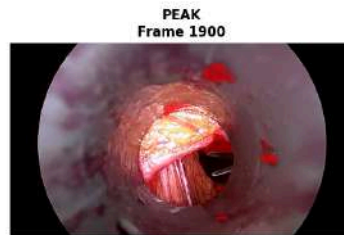
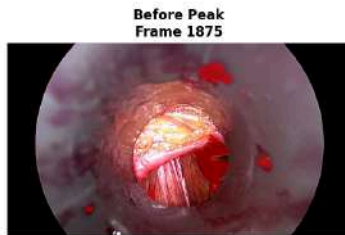
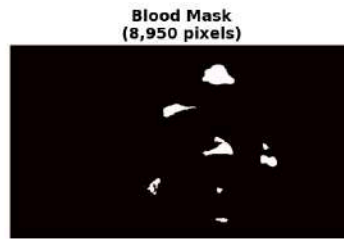
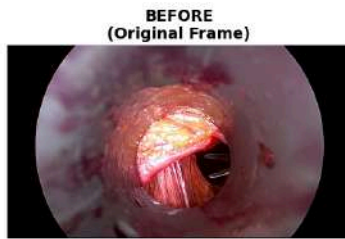
**Video 01 - Peak 27 at Frame 41900
Maximum Bleeding Event**



✓ Peak 27: Frame 41900 (11,143 pixels)

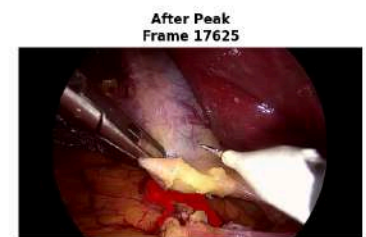
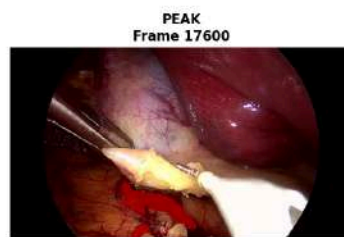
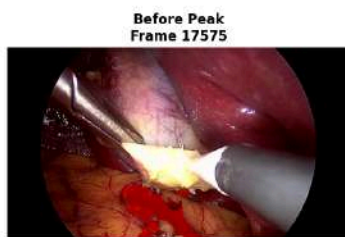
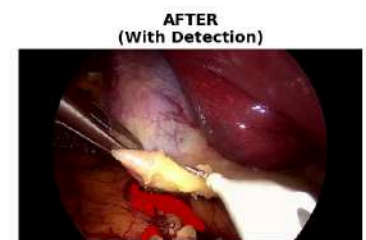
Video 02: Visualizing 34 peaks...

**Video 02 - Peak 1 at Frame 1900
Maximum Bleeding Event**



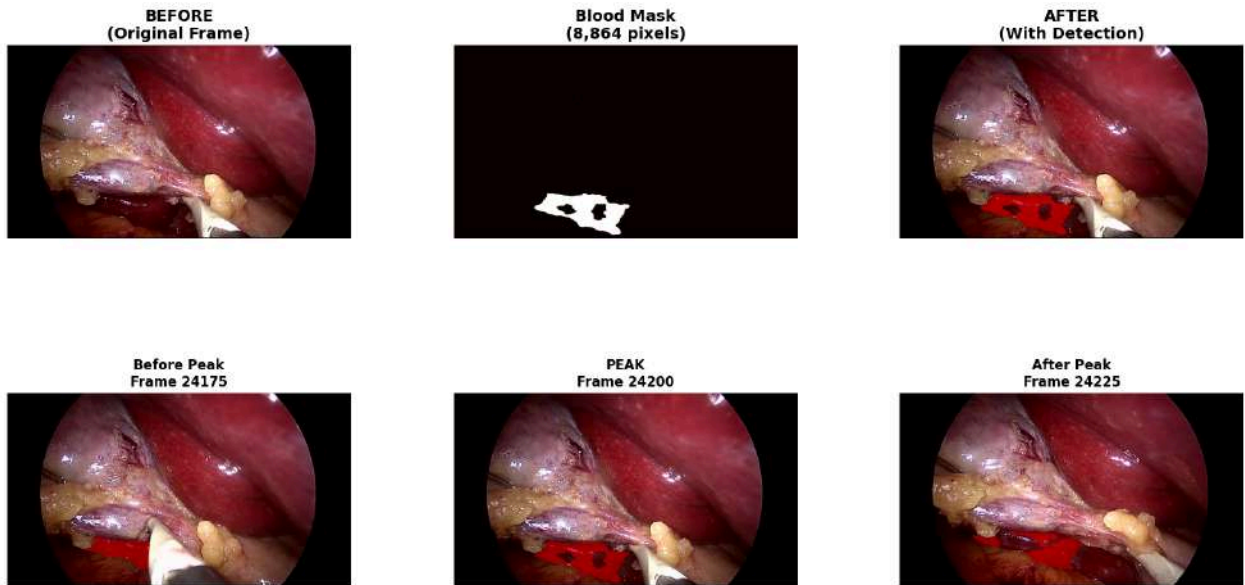
✓ Peak 1: Frame 1900 (8,950 pixels)

**Video 02 - Peak 2 at Frame 17600
Maximum Bleeding Event**



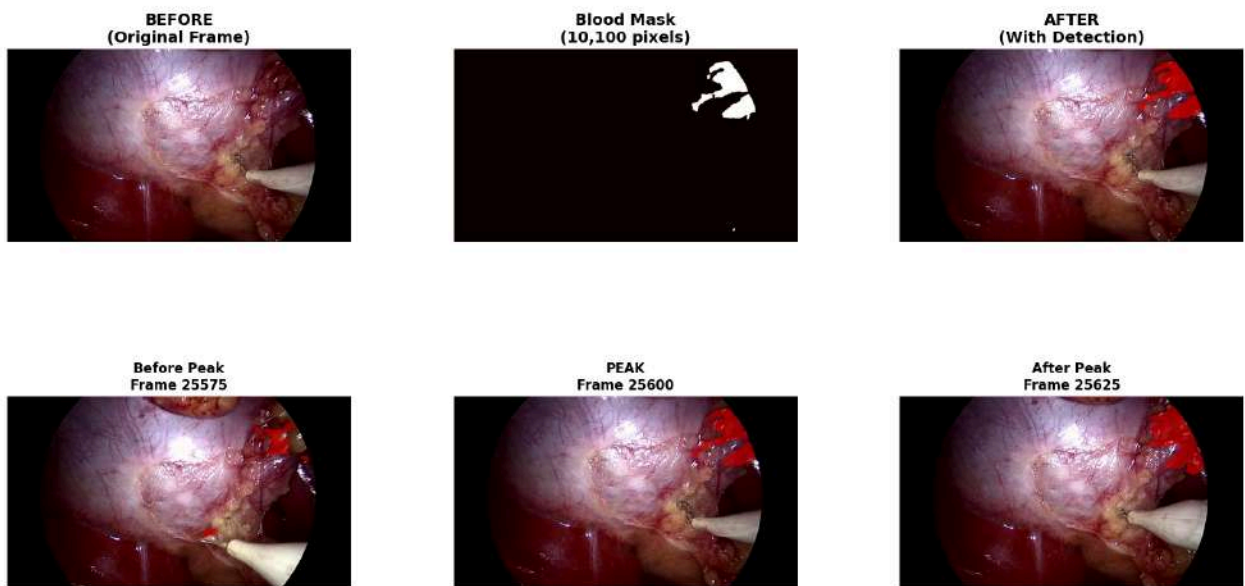
✓ Peak 2: Frame 17600 (9,994 pixels)

Video 02 - Peak 3 at Frame 24200
Maximum Bleeding Event



✓ Peak 3: Frame 24200 (8,864 pixels)

Video 02 - Peak 4 at Frame 25600
Maximum Bleeding Event



✓ Peak 4: Frame 25600 (10,100 pixels)



PART 8: Generate Comprehensive Report

```
In [ ]: # Cell 14: Generate Comprehensive Statistics Report
print("📄 Generating comprehensive report...\n")

# Compile statistics
```

```

report_data = []

for video_id in video_frames.keys():
    frames = video_frames[video_id]
    blood_areas = video_blood_areas[video_id]
    smoothed = video_blood_areas_smooth[video_id]
    peaks = video_peaks[video_id]
    confidences = video_confidences[video_id]

    # Calculate statistics
    blood_frames = [a for a in blood_areas if a > CONFIG['min_blood_area']]

    stats = {
        'video_id': video_id,
        'total_frames': len(frames),
        'frames_with_blood': len(blood_frames),
        'blood_percentage': 100 * len(blood_frames) / len(frames) if len(frames) > 0 else 0,
        'mean_blood_area': np.mean(blood_frames) if blood_frames else 0,
        'median_blood_area': np.median(blood_frames) if blood_frames else 0,
        'max_blood_area': np.max(blood_areas),
        'total_blood': np.sum(blood_areas),
        'num_bleeding_events': len(peaks),
        'mean_confidence': np.mean([c for c in confidences if c > 0]) if any(c > 0 for c in confidences) else 0,
        'duration_minutes': len(frames) / CONFIG['fps'] / 60
    }

    # Peak statistics
    if len(peaks) > 0:
        peak_heights = smoothed[peaks]
        stats['mean_peak_height'] = np.mean(peak_heights)
        stats['max_peak_height'] = np.max(peak_heights)
        stats['peak_frames'] = peaks.tolist()
    else:
        stats['mean_peak_height'] = 0
        stats['max_peak_height'] = 0
        stats['peak_frames'] = []

    report_data.append(stats)

# Create DataFrame
df = pd.DataFrame(report_data)

# Display
print("\n" + "="*80)
print("TEMPORAL BLOOD TRACKING REPORT")
print("="*80)

for _, row in df.iterrows():
    print(f"\nVideo {row['video_id']:02d}:")
    print(f"    Duration: {row['duration_minutes']:.1f} minutes")
    print(f"    Total frames: {row['total_frames']}")
    print(f"    Frames with blood: {row['frames_with_blood']} ({row['blood_percentage']:.1f}%)")
    print(f"    Mean blood area: {row['mean_blood_area']:.0f} pixels")

```

```

print(f" Max blood area: {row['max_blood_area']:,} pixels")
print(f" Total blood: {row['total_blood']:,} pixel-frames")
print(f" Bleeding events: {row['num_bleeding_events']}")
if row['num_bleeding_events'] > 0:
    print(f" Mean peak height: {row['mean_peak_height']:,} pixels")
    print(f" Max peak height: {row['max_peak_height']:,} pixels")
print(f" Avg confidence: {row['mean_confidence']:.3f}")

print(f"\n" + "="*80)

# Save to CSV
df.to_csv(OUTPUT_DIR / 'results/blood_tracking_summary.csv', index=False)
print(f"\n✅ Summary saved: {OUTPUT_DIR / 'results/blood_tracking_summary.csv'}")

# Save detailed results to JSON
detailed_results = {
    'config': CONFIG,
    'videos': {}
}

for video_id in video_frames.keys():
    detailed_results['videos'][f'video{video_id:02d}'] = {
        'blood_areas': video_blood_areas[video_id],
        'blood_areas_smooth': video_blood_areas_smooth[video_id].tolist(),
        'peaks': video_peaks[video_id].tolist(),
        'confidences': video_confidences[video_id],
        'statistics': report_data[[r['video_id'] for r in report_data].index(v
    }

with open(OUTPUT_DIR / 'results/detailed_tracking_data.json', 'w') as f:
    json.dump(detailed_results, f, indent=2)

print(f"\n✅ Detailed results saved: {OUTPUT_DIR / 'results/detailed_tracking_da

```

```

In [ ]: # Cell 15: Create Summary Visualization
print("\n📊 Creating summary visualization...\n")

# Create comprehensive summary figure
fig = plt.figure(figsize=(20, 12))
gs = fig.add_gridspec(3, 3, hspace=0.3, wspace=0.3)

# 1. Total blood per video (bar chart)
ax1 = fig.add_subplot(gs[0, 0])
video_ids_list = list(video_blood_areas.keys())
totals = [df[df['video_id']==vid]['total_blood'].values[0]/1000 for vid in video_ids_list]
ax1.bar(range(len(video_ids_list)), totals, color=colors[:len(video_ids_list)])
ax1.set_xticks(range(len(video_ids_list)))
ax1.set_xticklabels([f'V{vid:02d}' for vid in video_ids_list])
ax1.set_ylabel('Total Blood (x1000 pixel-frames)')
ax1.set_title('Total Blood Loss', fontweight='bold')
ax1.grid(True, alpha=0.3, axis='y')

# 2. Number of bleeding events

```

```

ax2 = fig.add_subplot(gs[0, 1])
events = [df[df['video_id']==vid]['num_bleeding_events'].values[0] for vid in video_ids_list]
ax2.bar(range(len(video_ids_list)), events, color=colors[:len(video_ids_list)])
ax2.set_xticks(range(len(video_ids_list)))
ax2.set_xticklabels([f'V{vid:02d}' for vid in video_ids_list])
ax2.set_ylabel('Number of Events')
ax2.set_title('Bleeding Events Detected', fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

# 3. Blood percentage
ax3 = fig.add_subplot(gs[0, 2])
percentages = [df[df['video_id']==vid]['blood_percentage'].values[0] for vid in video_ids_list]
ax3.bar(range(len(video_ids_list)), percentages, color=colors[:len(video_ids_list)])
ax3.set_xticks(range(len(video_ids_list)))
ax3.set_xticklabels([f'V{vid:02d}' for vid in video_ids_list])
ax3.set_ylabel('Percentage (%)')
ax3.set_title('% of Frames with Blood', fontweight='bold')
ax3.grid(True, alpha=0.3, axis='y')

# 4-6. Individual video curves
for i, video_id in enumerate(video_ids_list):
    ax = fig.add_subplot(gs[1 + i//3, i%3])

    smoothed = video_blood_areas_smooth[video_id]
    peaks = video_peaks[video_id]
    time_points = np.arange(len(smoothed)) / CONFIG['fps'] / 60

    ax.plot(time_points, smoothed, color=colors[i], linewidth=3)

    if len(peaks) > 0:
        ax.plot(time_points[peaks], smoothed[peaks], 'r*', markersize=15)

    ax.set_xlabel('Time (minutes)')
    ax.set_ylabel('Blood Area (pixels)')
    ax.set_title(f'Video {video_id:02d} - Temporal Profile', fontweight='bold')
    ax.grid(True, alpha=0.3)

fig.suptitle('Temporal Blood Tracking - Comprehensive Summary', fontsize=18, fontweight='bold')
plt.savefig(OUTPUT_DIR / 'visualizations/comprehensive_summary.png', dpi=300)
plt.show()

print("✅ Summary visualization saved!")
print(f"    File: {OUTPUT_DIR / 'visualizations/comprehensive_summary.png'}")

```

✅ COMPLETE! Summary

This notebook successfully:

1. ✅ Loaded trained UNet++ model from Video01 training

2. ☒ Processed 3 Cholec80 videos (extracted frames at 1 FPS)
3. ☒ Generated blood masks for all frames
4. ☒ Created BEFORE/AFTER visualizations
5. ☒ Calculated blood area with industry-grade methods
6. ☒ Plotted temporal blood curves (all 3 videos)
7. ☒ Detected bleeding events using peak detection
8. ☒ Visualized peak frames with context
9. ☒ Generated comprehensive statistics

Outputs:

- Blood masks: `temporal_blood_tracking/masks/`
- Visualizations: `temporal_blood_tracking/visualizations/`
- Results: `temporal_blood_tracking/results/`

Next Steps for ISBI 2026:

- Use blood curves for blood loss estimation
- Correlate peaks with surgical events
- Compare with clinical ground truth
- Generate paper figures!



Ready for publication!